

CMP514 – Computer Graphics and Image Processing Laboratory

B Tech CSE TY

Activity 4 – Line drawing using DDA algorithm in C Graphics (graphics.h)

Aim: To implement and analyze two variants of the Digital Differential Analyzer (DDA) line-drawing algorithm, comparing their rounding mechanisms, loop structures, and execution safety under edge cases.

Mathematical Foundations & Algorithms

Variant 1 - Basic Rounded DDA

Loop Bounds: $i = 0$ to $i < \text{steps}$ (Exactly steps iterations)

```
1. Read the input of two end points (X1,Y1) and (X2,Y2)
2. Calculate the difference between two end points
   dx = X2 - X1
   dy = Y2 - Y1
3. Identify the number of steps based on the calculation in step 2
   if dx > dy,
       then more steps in x coordinate;
   else
       More steps in y coordinate.

   if (absolute(dx) > absolute(dy))
       steps = absolute(dx);
   else
       steps = absolute(dy);
4. Calculate the increment in x coordinate and y coordinate.
   Xincrement = dx / (float) steps;
   Yincrement = dy / (float) steps;
5. Initialize x=x1 and y=y1
```

```

6. Put the pixel by incrementing x and y coordinates accordingly
   for (i=0; i < Steps; i++)
   {
       putpixel(Round(x), Round(y));
       x = x + Xincrement;  y = y + Yincrement;
   }
7. Stop

```

Variant 2 - Optimized Sign-Bias DDA

Loop bounds: step = 0 to step <= length (Exactly length + 1 iterations)

```

1. Read end points (x1, y1) and (x2, y2).
   If x1 = x2 and y1 = y2.
       Plot Point.
       Exit.
2.  $\Delta x = |x_2 - x_1|$  and  $\Delta y = |y_2 - y_1|$ 
3. If ( $\Delta x \geq \Delta y$ )
       length =  $\Delta x$  else
       length =  $\Delta y$ 
4.  $\Delta x = (x_2 - x_1) / \text{length}$  ,
    $\Delta y = (y_2 - y_1) / \text{length}$ 
5. Calculate
    $x = x_1 + 0.5 * \text{Sign}(\Delta x)$  and
    $y = y_1 + 0.5 * \text{Sign}(\Delta y)$ 
6. Loop for step=0, step  $\leq$  length, step =step+1
   {
       plot (Integer (x), Integer (y))
        $x = x + \Delta x$ ,  $y = y + \Delta y$ 
   }

```

7. Stop

3. Lab Assignment and Observations

Step 1: Trace analysis task

Compute and compile trace tables and display the output in text mode

Expected Trace Table Structure:

- **Variant 1:** Columns for `i`, `Current X`, `Current Y`, `Round(X)`, `Round(Y)`, `Plotted Pixel`.
- **Variant 2:** Columns for `Step`, `Current X`, `Current Y`, `(int)X`, `(int)Y`, `Plotted Pixel`.

Step 2: Visualization verification

Plot the lines using both variants – use different colors to plot these lines and observe the non-overlapping pixels.

Step 3: Benchmarking

While both algorithms have a linear **time complexity of $O(N)$** relative to the number of pixels drawn, their performance differs at the microarchitecture level. Calling custom mathematical rounding logic (`Round()`) inside a loop introduces computational branching overhead. In contrast, integer truncation (`(int)`) leverages fast execution paths directly inside modern CPU arithmetic logic units (ALUs).

To benchmark this execution difference, we introduce a performance stress test that measures system execution clock cycles while plotting a dataset of **1,000 distinct lines**.

Metric Parameter	Variant 1 (Basic Rounded)	Variant 2 (Sign-Bias Optimized)
Total lines processed	1000 Lines	1000 Lines
Total microseconds	<i>[Enter Log]</i>	<i>[Enter Log]</i>
Average time per line	<i>[Total Time / 1000]</i>	<i>[Total Time / 1000]</i>
Calculated speed factor increase	Baseline (1.0)	<i>[Time_{V1} / Time_{V2}]</i>

Step 4: Programming Challenge

To deepen your understanding of structural decomposition, you will break away from incremental stepping and implement line drawing using alternative computational structures.

The Recursive DDA Approach

Instead of updating an iterative loop counter, a recursive line drawer tracks line segments by passing the fractional floating-point variables down a call stack until it crosses the structural boundary limit.

- **Base Case:** If the structural loop depth matches or exceeds the required coordinate step bounds, stop execution and unwind the stack.
- **Recursive Step:** Plot the current coordinates using the chosen rounding method, apply the calculated increments directly to the parameters, and pass the updated states into the next recursive functional branch step.

The Divide-and-Conquer Approach (Midpoint Subdivision)

Instead of walking sequentially from start to end, a divide-and-conquer strategy bisects space. It finds the structural midpoint between two target endpoints, plots that midpoint pixel, and splits the line segment into two independent sub-segments.

- **Base Case:** If the distance between two coordinate inputs drops below a single pixel radius (e.g., $\max(|x_2 - x_1|, |y_2 - y_1|) \leq 1$), plot the endpoints and exit.
- **Conquer Step:** Calculate the geometric midpoint coordinates:
 $x_{mid} = (x_1 + x_2) / 2$, and $y_{mid} = (y_1 + y_2) / 2$
- **Divide Step:** Concurrently dispatch two independent sub-tasks to process the left half and the right half of the line:
 1. `drawSubdivision(x1, y1, x_mid, y_mid)`
 2. `drawSubdivision(x_mid, y_mid, x2, y2)`

Step 5 Architectural Comparison Matrix

- Students must fill out the performance profile tracking table below after executing all four methods.

Algorithm Paradigm	Time Complexity	Memory/Stack Overhead	Risk Factors / Structural Limits
Iterative basic rounded	$O(N)$	$O(1)$ (Constant)	High precision drop error, Division by Zero
Iterative sign-bias optimized	$O(N)$	$O(1)$ (Constant)	None (Robust Baseline)
Recursive state-step	$O(N)$	<i>[To deduce]</i>	Stack Overflow on extremely long lines
Divide-and-Conquer (subdivision)	<i>[To deduce]</i>	<i>[To deduce]</i>	Duplicate pixel rendering overhead